What Does a Benign-Trained Anomaly Detector Actually Detect?

Suramya Angdembay *suramya.angdembay@usm.edu*
University of Southern Mississippi

Haiyan Tian *haiyan.tian@usm.edu*
University of Southern Mississippi

Abstract

Anomaly detectors for insider threats are usually judged by how well their scores rank malicious activity above normal activity. We show that this number can be misleading, because it does not reveal what information the detector uses. We build a detector by adapting a large language model to normal employee activity only, so that unusual activity receives a high surprisal score, and we then open the model to determine what its score is based on. On one standard benchmark the score turns out to rest almost entirely on who each user is (their recorded personality and organizational profile) rather than on what they did; on a second benchmark the score rests mostly on behavior. When malicious users are compared only against normal users that the model has never seen, apparent ranking quality falls from about 0.95 to near chance on the first benchmark. The same analysis on a third dataset, collected from real people with real personality questionnaires, confirms the behavioral case, and classical statistical detectors trained on the same information do not take the profile shortcut. The conclusion is that high ranking performance alone does not show that a detector has learned to detect behavior, and we provide a way to check.

1 The problem

Organizations record what employees do on their computers: logons, file transfers, web activity, e-mail volume, and so on. An *insider threat* is an employee who misuses legitimate access, for example by stealing data. Because such events are rare, a common approach is *anomaly detection*: build a statistical model of normal activity, and flag days that the model finds unusual.

Formally, a detector is a function s that assigns a real-valued *anomaly score* $s(x)$ to each observation x, where in our setting x is a record of one user's activity on one day. Higher scores are meant to indicate more suspicious activity. Detectors are usually evaluated by how well their scores *rank* malicious days above benign ones. The standard summary of ranking quality is the following probability, estimated over a labeled test set:

$$\mathrm{AUC} \;=\; \Pr\big(s(X_{\mathrm{mal}}) > s(X_{\mathrm{ben}})\big), \tag{1}$$

where X_{mal} is a randomly chosen malicious day and X_{ben} a randomly chosen benign day. (This quantity is commonly called the ROC-AUC, for "area under the receiver operating characteristic curve"; the probability formulation above is equivalent and is the only form we need.) An AUC of 1 means perfect ranking and 0.5 means the ranking is no better than chance.

The question that motivates our work is simple to state. A high AUC says that the score *separates* the two classes. It does not say *on the basis of what information* the separation is achieved. Two detectors with identical AUC could work very differently: one might respond to genuinely unusual behavior, while the other might respond to incidental properties of *who* the malicious users happen to be, such as their role, department, or personality profile. The second kind of detector would be nearly useless in practice, because in deployment the malicious users are not known in advance. Our paper develops a method for determining

which kind of detector one actually has, and applies it to a modern detector built from a large language model.

2 The data

We use two versions (called r4.2 and r6.2) of the CERT insider-threat corpus, a widely used synthetic dataset that simulates a corporation's activity logs, produced by Carnegie Mellon University. Each version contains several thousand simulated employees observed over about eighteen months, with a small set of employees scripted to carry out malicious scenarios on particular days. Version r4.2 has sixty malicious users; version r6.2 has four. We later add a third dataset, TWOS, collected from real people; it is described in Section 5.

Each user-day is converted into a short structured text with three kinds of lines:

- a `DAY` line holding static organizational fields for that user (project, role, business unit, department, team, and an information-technology administrator flag);
- a `PSY` line holding the user's five simulated personality scores. These are the "Big Five" personality traits used in psychology: openness (O), conscientiousness (C), extraversion (E), agreeableness (A), and neuroticism (N). In CERT these numbers are generated by the simulator; they are constant for each user;
- one `SES` line per work session, holding numeric behavioral counts for that session: duration, number of logons, file and removable-drive activity, e-mail volume, web-browsing category counts, and similar quantities.

A user-day text therefore looks like a small table serialized as text. The first two lines describe *who the user is* and are essentially constant across a user's days; the remaining lines describe *what the user did* that day.

3 The detector

3.1 Language models as probability distributions

A *language model* is a parametric probability distribution over sequences of symbols called tokens. Writing a sequence as $x = (x_1, \ldots, x_n)$, the model has the product form

$$p_\theta(x) \;=\; \prod_{t=1}^{n} p_\theta\big(x_t \mid x_1, \ldots, x_{t-1}\big), \tag{2}$$

where each conditional distribution is computed by a neural network with parameters θ. Modern "large" language models are such networks with billions of parameters, trained on enormous general text corpora. We use one called Qwen3-8B (eight billion parameters). The network processes a sequence through L successive layers; at layer ℓ and position t it maintains an internal state vector $h_t^{(\ell)} \in \mathbb{R}^d$ with $d = 4096$. These internal vectors matter later: they are the objects we will analyze.

3.2 Adapting the model to normal behavior only

We specialize the general-purpose model to our corpus by *fine-tuning*: continuing its training on the serialized user-day texts, using only days from benign users. The training objective is maximum likelihood, that is, minimizing the negative log-likelihood of the benign corpus $\mathcal{D}$:

$$\mathcal{L}(\theta) \;=\; -\sum_{x \in \mathcal{D}} \sum_{t} \log p_\theta\big(x_t \mid x_{<t}\big). \tag{3}$$

No malicious example is ever used in training. This is the classical one-class setup: model the normal population, then measure deviation from it.

For efficiency the fine-tuning does not modify all eight billion parameters. Each weight matrix W of the network is kept fixed and receives a trainable low-rank correction,

$$W' \;=\; W + \tfrac{\alpha}{r} BA, \qquad B \in \mathbb{R}^{d_{\text{out}} \times r},\; A \in \mathbb{R}^{r \times d_{\text{in}}}, \tag{4}$$

with rank $r = 16$, so only the small matrices A and B are learned. This technique is called LoRA (low-rank adaptation); we use a memory-saving variant called QLoRA in which the fixed weights are stored at reduced numerical precision. The pair (fixed base model, learned low-rank correction) is called the *adapted model*. The important point is that the entire effect of training is carried by a low-dimensional, removable perturbation, which makes "before versus after" comparisons clean.

3.3 The anomaly score

The score of a day x is its average surprisal under the adapted model:

$$s(x) \;=\; -\frac{1}{n} \sum_{t=1}^{n} \log p_{\theta_{\text{ada}}}\big(x_t \mid x_{<t}\big). \tag{5}$$

Days that the model of normal behavior finds improbable receive high scores. On the standard evaluation protocol for these benchmarks the score ranks very well: AUC about 0.95 on r6.2 and 0.96 on r4.2. By the usual standards of the field, this would be reported as a strong detector. The rest of this document asks what that number is actually made of.

4 Looking inside the model

4.1 What did fine-tuning change?

Because the base model is kept frozen, we can run the same day x through both the base and adapted models and subtract their internal states. Define, at a chosen layer ℓ and each token position t,

$$\delta_t \;=\; h_t^{(\ell),\text{ada}} - h_t^{(\ell),\text{base}} \;\in\; \mathbb{R}^d. \tag{6}$$

The vector δ_t is precisely the contribution of the fine-tuning to the model's internal computation at that point. Everything the detector learned about our corpus, beyond what the general-purpose model already knew, must be expressed through these vectors.

4.2 A sparse dictionary for the changes

The vectors δ_t live in $\mathbb{R}^{4096}$ and are not individually interpretable. We therefore approximate them by sparse nonnegative combinations of learned dictionary elements. Concretely, we seek a dictionary matrix $D \in \mathbb{R}^{d \times m}$ (with $m = 4d$ columns, called *features*) and, for each δ_t, a coefficient vector $z_t \in \mathbb{R}^m_{\geq 0}$ with at most $k = 8$ nonzero entries, such that

$$\delta_t \;\approx\; D z_t, \tag{7}$$

with D and the code map chosen to minimize the mean squared reconstruction error over a large sample of token positions. This is a sparse coding problem; in the machine-learning literature the particular parametrization we use (a one-layer encoder network followed by a keep-the-largest-k rule and a linear decoder) is called a *sparse autoencoder*. The outcome is that each change vector is expressed as a short combination of reusable directions, and each direction can be studied on its own.

4.3 Selecting candidate features

Some dictionary features activate similarly on malicious and benign days; those cannot carry the anomaly signal. We rank features by the difference in mean activation,

$$g_f \;=\; \mathbb{E}\big[z_{t,f} \mid \text{malicious day}\big] - \mathbb{E}\big[z_{t,f} \mid \text{benign day}\big], \tag{8}$$

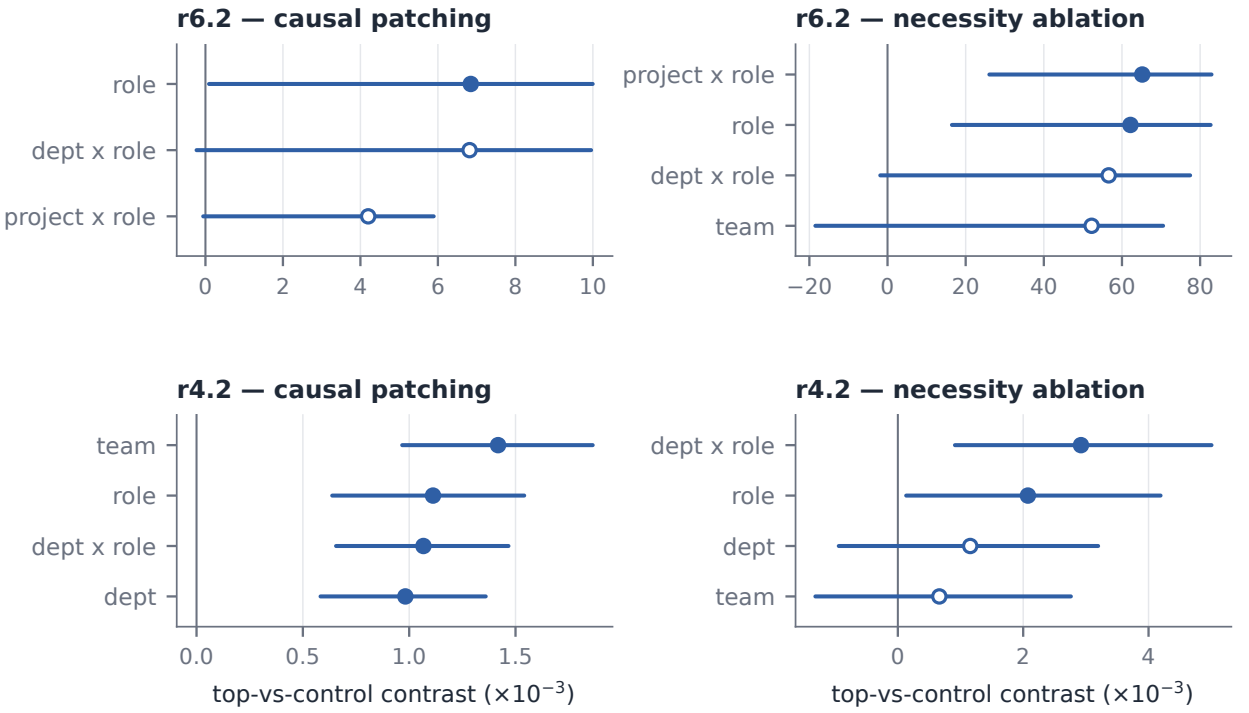

Figure 1: Effect of editing the selected features, compared with editing matched control features, on the anomaly score (one panel per benchmark and per type of edit; units are thousandths of the score). Each point is the average effect for one organizational matching context, and each horizontal bar is a 95 percent confidence interval computed with the malicious users as the resampling unit. Positive values mean that the selected features influence the score more strongly than the control features do. Open circles mark intervals that include zero.

and take the five features with the largest g_f as the candidate mechanism. The full paper verifies, through intervention experiments, that these selected features genuinely influence the anomaly score: if one edits the coefficients $z_{t,f}$ of the selected features and propagates the edited internal state through the rest of the network, the score changes, and it changes more than when the same edit is applied to matched control features. We omit the details of those experiments here and take away only their conclusion: the selected features are causally connected to the score, not merely correlated with it. Figure 1 summarizes those measurements.

4.4 Reading off what the features respond to

Finally we ask *where* the selected features activate within the day text. Every token position belongs to one line class: the organizational `DAY` line, the personality `PSY` line, or a behavioral `SES` line. For a feature f, define its *attribution mass* on class c as the fraction of its total activation that occurs at positions of that class:

$$m_f(c) \;=\; \frac{\sum_{(x,t)\,:\,\mathrm{class}(t)=c} z_{t,f}(x)}{\sum_{(x,t)} z_{t,f}(x)}. \tag{9}$$

If a feature has nearly all of its mass on `PSY` and `DAY` positions, it responds to who the user is; if its mass lies on `SES` positions, it responds to what the user did. This single quantity is what turns the analysis into an answer to our opening question.

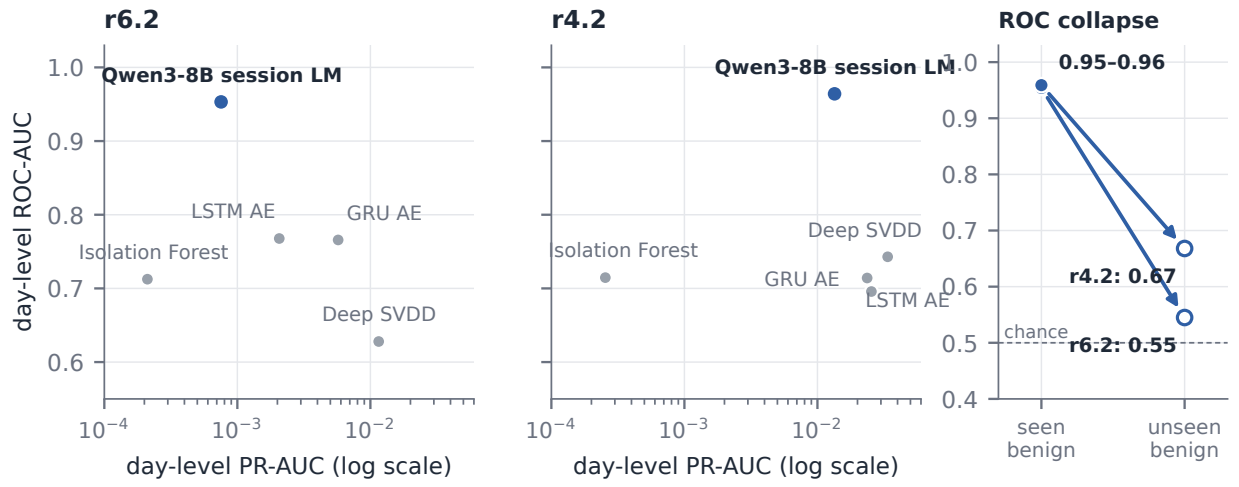

Figure 2: Score-level results. Left and center panels: ranking quality (vertical axis) and precision (horizontal axis, logarithmic scale) for the language-model detector (blue) and four standard anomaly-detection methods on the two benchmarks; the language model attains the highest ranking quality under the standard protocol. Right panel: the same language-model ranking quality when malicious users are compared only against benign users never seen during training. The apparent advantage largely disappears (from 0.95 to 0.55 on r6.2 and from 0.96 to 0.67 on r4.2), and the dashed line marks chance level.

5 Findings

5.1 Two detectors that look alike and are not

The two CERT versions produce nearly identical headline AUC values, yet the analysis above reveals qualitatively different detectors.

On r6.2, the five selected features place between 99.8 and 100 percent of their attribution mass on the personality and organizational lines. The detector's causally relevant evidence is almost entirely a description of *who the user is*. On r4.2, four of the five selected features concentrate on the behavioral session lines. There, the evidence is mostly *what the user did*.

5.2 The consequence for the headline numbers

Why would reading a user's static profile produce a high AUC at all? The adapted model was trained on roughly ninety percent of the benign users. For those users, the model has memorized the constant profile lines, so their days look unsurprising. The malicious users, however, were never seen in training, so their profile lines alone make their days look surprising. The score is then partly measuring *familiarity* rather than behavior.

This suggests a sharper test: compare malicious days only against benign days of users who were also excluded from training, so that both sides are equally unfamiliar. Under this comparison the AUC falls from 0.95 to 0.55 on r6.2, which is barely better than chance, and from 0.96 to 0.67 on r4.2. The apparently excellent detector on r6.2 was, to a first approximation, an unfamiliar-user detector. Consistently with this, deleting the personality lines from the inputs at scoring time *improves* the r4.2 detector's performance on unseen users from 0.67 to about 0.80: the profile text was not merely unhelpful, it was actively obscuring the behavioral signal. Figure 2 shows the comparison with standard detectors and the collapse.

5.3 A real-data test with real personality scores

The CERT data are simulated, including the personality scores, so we repeated the entire pipeline on TWOS, a dataset collected from twenty-four real users over one week, during which some users, by design of the underlying study, temporarily took over other users' sessions. Each real user completed a standard fifty-item Big Five personality questionnaire, so the serialized days contain genuine psychometric information. In this dataset half of the users experience an attack, so "being an unusual person" cannot separate the classes, and our account predicts that the detector must rely on behavior. That is what the analysis finds: the selected features place essentially all of their mass on the behavioral lines, under two independent training runs, and the score meaningfully separates each user's attacked windows from that same user's normal windows (average within-user AUC about 0.7, with identity held fixed by construction).

5.4 Where the shortcut comes from

One might suspect that the r6.2 data simply force any statistical method to use profile information. This is testable. We trained three classical anomaly detectors (an isolation forest, a one-class support vector machine, and a principal-component reconstruction method; their details do not matter here) on the same days, represented as ordinary numeric feature vectors that *include* the profile variables. All three essentially ignore the profile variables: those variables receive about four to seven percent of the total importance, no profile variable appears among any method's top five, and deleting the profile variables does not hurt their performance.

The shortcut is therefore specific to the language-model pathway, and the reason is visible in the training objective. For a likelihood-based sequence model, the per-user-constant profile tokens are the most predictable part of every training sequence, so memorizing them yields the largest and cheapest reduction of the training loss. In a plain feature-vector representation, the same information is twelve columns among two hundred forty-two, with no comparable incentive attached. Our summary of the whole phenomenon is a two-factor statement: the population structure of a dataset determines *when* an identity shortcut is available, and the training objective together with the data representation determines *which* detectors take it.

6 Why this matters

Three conclusions carry beyond these particular datasets and this particular model.

First, for anomaly detectors intended to detect behavior, ranking quality alone is not evidence of success. A detector should be evaluated with the unfamiliarity confound removed, by comparing malicious observations only against normal observations from equally unseen users, and, where possible, by within-user comparisons.

Second, including static descriptors of people (role, department, personality measures) in a detector's input is not harmless. In the likelihood-based detector it created the shortcut, and even where the detector was behavioral it degraded performance.

Third, the question "what does this detector actually detect?" can be answered, not merely asked. The chain presented here, namely differences of internal states, a sparse dictionary approximation, selection of score-relevant directions, and attribution of those directions to input content, turns the question into a measurable quantity. In the full paper this chain is supported by causal intervention experiments and an extensive robustness program, and it is that combination, rather than any single number, that justifies the conclusions summarized here.